

Practical Techniques in Fundamental Software Development: Intelligent Software Engineering

Intelligent Software Engineering Experiment Report

Experiment 2: Machine Learning-based Code Review for Human-Written Code

Name	Marlen Melis	Department	School of Computing
Student ID	2026140428	Course Teacher	Liu Jiakun
Instructor	Liu Jiakun	Lab Location	Dormitory
Lab Time	Evening		
Lab Score		Discussion Score	
Report Score		Total Experiment Score	

1 Experiment Objectives

Upon completion of this experiment, I am able to:

1. Understand the basic workflow of traditional machine learning in code review tasks: data filtering → feature extraction → feature-vector construction → model training → merge prediction.
2. Apply data-preprocessing methods specific to code review tasks, including handling partially-missing structural coverage honestly (rather than silently imputing a fake value) and building a leakage-aware, temporally-tiered feature specification.
3. Construct Abstract Syntax Trees (ASTs) and Control Flow Graphs (CFGs) for code, using both a maintained, real-CFG toolchain (Python) and a lightweight

tree-sitter-based approximation (TypeScript/C#), and state honestly where each approach’s fidelity actually breaks down.

4. Extract structural, textual, and code-modification features from Pull Request data, while verifying by direct inspection of the lab guide (not a paraphrase of it) exactly which features are and are not permitted.
5. Train and grid-search two traditional machine learning algorithms — Support Vector Machines (RBF kernel) and Random Forest — for a binary merge-prediction task.
6. Evaluate the resulting models with Accuracy/Precision/Recall/F1/ROC-AUC, explicitly surfacing minority-class (not-merged) recall, analyze Random Forest feature importance by category, and compare model and feature-set variants against each other with real held-out numbers rather than a single reported figure.

2 Experiment Content

1. Data Filtering

The Experiment 1 dataset (1,494 mined PRs) was filtered to **human-written** (`is_ai_authored == False`) and **closed** (`state ∈ {MERGED, CLOSED}`) PRs, per the lab guide’s own Step 1. Every mined PR in this dataset is already closed, so the second half of the filter is a documented no-op rather than an unstated assumption. This yields a **training population of 1,114 PRs**.

2. Feature Specification and the Leakage Design

The full specification lives in `reports/exp2_feature_spec.md`. Before writing any feature-extraction code, every candidate feature was tagged with *when its value becomes known* relative to the review process: T0 (submission-time, e.g. title/body text), T1 (final-code-state — known once the code is final, but the stored diff already reflects any commits pushed in response to review), and T2 (review-process outcome — review/comment counts, approval state; an `APPROVED` review essentially *is* the merge decision). Reading the actual lab guide PDF directly (rather than trusting a paraphrase) surfaced a real discrepancy: the project plan’s own draft claimed the guide’s Step 4 lists `num_reviewers/num_comments` as Experiment-2 features. In the real PDF, those numbers belong to **Experiment 1’s** descriptive-statistics list, and Experiment 2’s

own feature list (§2.7.3) never included them — §2.4.3 explicitly forbids review comments/discussion as features. This produced three feature variants rather than the two originally planned:

- **V1 (per-spec)** — code-structure (S) $\cup$ code-modification (M) $\cup$ textual (T), exactly §2.7.3/§2.4.3. The compliant, headline feature set.
- **V2 (pre-review-strict)** — V1 minus the two features that keep growing during review (`m_num_commits`, `t_commit_msg_len_total`), plus `author_prior_merge_rate/author_prior_pr_count`, computed only from PRs the same author closed *strictly before* this PR, scoped to the same repository.
- **V0 (leaky superset, ablation only)** — V1 plus the forbidden process/review features (`p_num_reviews`, `p_has_approved`, ...) and `num_labels`. Never a compliant result — built solely to *measure* how much these signals inflate apparent accuracy, rather than asserting it.

3. AST/CFG Intermediate Representations

Implemented in `src/features/ast_cfg.py`. Since Experiment 1 mined unified-diff patches rather than full before/after file snapshots, every AST/CFG here is built from the **hunk-level reconstructed post-image text** (context lines plus added lines, removed lines dropped) — not the whole file, a scope limitation stated up front rather than discovered later. Two genuinely different tools were used, matching the guide’s own language split: **Python** (semantic-kernel, home-assistant) uses the stdlib `ast` module for a real AST and `staticfg` for a real CFG; **TypeScript** (vscode) and **C#** (runtime, aspnetcore) use `tree-sitter` grammars for a real AST, with the CFG approximated by counting `if/for/while/switch/try` decision nodes (D) and setting `cfg_node_count = D + 1`, `cfg_edge_count = 2D` — not a faithful graph, but exactly reproducing the standard McCabe cyclomatic-complexity value ($E - N + 2 = D + 1$).

4. Model Training

Implemented in `src/ml/train_svm.py` and `train_rf.py`. Data was split **per repository**, oldest $\sim 80\%$ to train and newest $\sim 20\%$ to test, rather than a single global time cutoff (see “Summary of Problems Encountered” for why the naive global cutoff was rejected). Support Vector Machines (RBF kernel, grid-searched $C \in \{0.1, 1, 10, 100\}$, $\gamma \in \{\text{scale}, \text{auto}, 0.01, 0.1\}$) and Random Forest (grid-searched `n_estimators` $\in \{100, 300, 500\}$, `max_depth` $\in \{\text{None}, 5, 10, 20\}$) were trained for all three feature variants, each compared

plain vs. `class_weight='balanced'` — 12 trained pipelines in total. Hyperparameter selection used `StratifiedKFold(5)` on the training split only (see Problems Encountered for why not `TimeSeriesSplit`), scored by ROC-AUC.

5. Model Evaluation

Implemented in `src/ml/evaluate.py`. Every saved pipeline was evaluated once, on the same reconstructed held-out test split it was trained against: Accuracy, per-class Precision/Recall/F1 (surfacing not-merged recall explicitly, since accuracy alone is inflated by the dataset’s 74/26 imbalance), and ROC-AUC. Random Forest’s built-in `feature_importances_` were mapped back to real column names and aggregated by category (S/M/T/P) rather than left as a flat per-feature ranking, to answer *which type of feature matters*, not just which single one.

3 Experimental Results

Requirement: Describe the results obtained from the experiment. Basic content should include:

1. Statistical Results of the Human-Written Code Dataset

Repo	Language	N	Merge rate
microsoft/vscode	TypeScript	195	85.1%
dotnet/runtime	C#	189	83.6%
home-assistant/core	Python	269	77.3%
dotnet/aspnetcore	C#	187	66.3%
microsoft/semantic-kernel	Python + C#	274	62.4%
Pooled	—	1,114	74.2%

827 merged / 287 not-merged (imbalance $\approx$ 2.88:1). The 23-point spread in merge rate across repos (62.4%–85.1%) means “which repo” is by itself a strong naive predictor — which is exactly why repo identity was deliberately excluded from V1/V2’s feature columns: the model should be judged on *code* features, not on memorizing which community it came from.

AST/CFG coverage. Only **71.8%** of these 1,114 PRs contain ≥ 1 file in a supported language (Python/TypeScript/C#); the remaining 28.2% (pure `.md/.json/.yaml/...diffs`) get `s_has_parseable_code=0` and all structure features honestly zeroed, never a guessed non-zero value.

2. Examples of AST and CFG Generation

Python hunk (real AST via `ast`, real CFG via `staticcfg`):

```
def foo(x):
    if x > 0:
        return x
    return -x

def bar():
    pass
```

Result: `ast_node_count=21, num_functions=2, cfg_node_count=5, cfg_edge_count=2, cyclomatic_proxy=3` (the bare `if` in `foo` contributes complexity 2; the branchless `bar` contributes 1; summed across the two independent function scopes). Changing only the condition to `if x:` (a bare-Name truthiness check, otherwise identical code) still parses fine but drives `cfg_node_count/cfg_edge_count/cyclomatic_proxy` to **0** — a real `staticcfg` bug traced to its exact root cause (`reports/exp2_ast_cfg_fidelity.md §4`), not merely observed.

A real mined PR, `microsoft/vscode#326961` (*“Fix chat list row height swallowed during render...”*, merged): `s_ast_node_count=3274, s_ast_max_depth=50, s_cfg_node_count=19, s_cfg_edge_count=18, s_cyclomatic_proxy=19, s_num_functions_touched=23` — generated by the tree-sitter/decision-node-counting path (TypeScript), across the 4 files this PR touched.

3. Examples of Feature Extraction Results

Same PR, `microsoft/vscode#326961`, full feature-extraction row:

Feature	Value	Feature	Value
m_additions	268	t_title_len	77
m_deletions	25	t_body_len	2,493
m_changed_files	4	t_commit_msg_len_first	1,087
m_num_hunks	10	s_ast_node_count	3,274
m_num_commits	5	s_cyclomatic_proxy	19

This row is exactly one line of `data/processed/features_v1.parquet`, joined from `pull_requests`, `files_changed`, `commits`, and Step 9’s AST/CFG cache — no review comments, reviewer counts, or discussion text anywhere in it.

4. Experimental Results — SVM

Variant	weight	Acc.	ROC-AUC	Rec(M)	Rec(NM)	F1(NM)
V0	plain	0.987	0.969	0.995	0.950	0.962
V0	balanced	0.987	0.984	0.995	0.950	0.962
V1	plain	0.817	0.561	0.973	0.100	0.163
V1	balanced	0.817	0.607	0.989	0.025	0.047
V2	plain	0.844	0.718	0.984	0.200	0.314
V2	balanced	0.835	0.740	0.967	0.225	0.327

5. Experimental Results — Random Forest

Variant	weight	Acc.	ROC-AUC	Rec(M)	Rec(NM)	F1(NM)
V0	plain	0.987	0.988	0.995	0.950	0.962
V0	balanced	0.982	0.989	0.989	0.950	0.950
V1	plain	0.821	0.678	0.946	0.250	0.333
V1	balanced	0.768	0.677	0.842	0.425	0.395
V2	plain	0.875	0.739	0.984	0.375	0.517
V2	balanced	0.826	0.744	0.913	0.425	0.466

weight = **class_weight**; **Rec(M)/Rec(NM)** = recall on the merged / not-merged class; **F1(NM)** = F1 on the not-merged class.

Held-out test set: 224 PRs (890 train), 82.1% merged — the per-repo split keeps every repo proportionally represented (55/54/39/38/38 across the 5 repos), unlike the rejected global cutoff.

6. Analysis of Feature Importance

Category	V0	V1	V2
P — process/forbidden	68.9%	—	—
M — code modification	14.6%	44.2%	37.9%
T — textual	11.8%	40.0%	32.2%
S — code structure	4.7%	15.8%	14.6%
Author history (V2 only)	—	—	15.3%

The single most important feature in V0 is **p_has_approved** (41.6% of all importance) — a concrete demonstration, not an assertion, of why the lab guide bans review-state features: an approval essentially *is* the merge decision. In the compliant variants, M and T dominate roughly evenly (no single feature or category quietly does all the work, unlike V0); S consistently contributes least, which is not a mystery but a direct consequence of Step 9’s own fidelity numbers — only 71.8% coverage, and even among parseable Python hunks **staticfg**’s CFG stage fails on $\sim 27\%$ of them. In V2, **author_prior_merge_rate** is individually the single *most* important feature (10.0%), more than any M or T feature alone.

7. Comparison of Performance Across Models

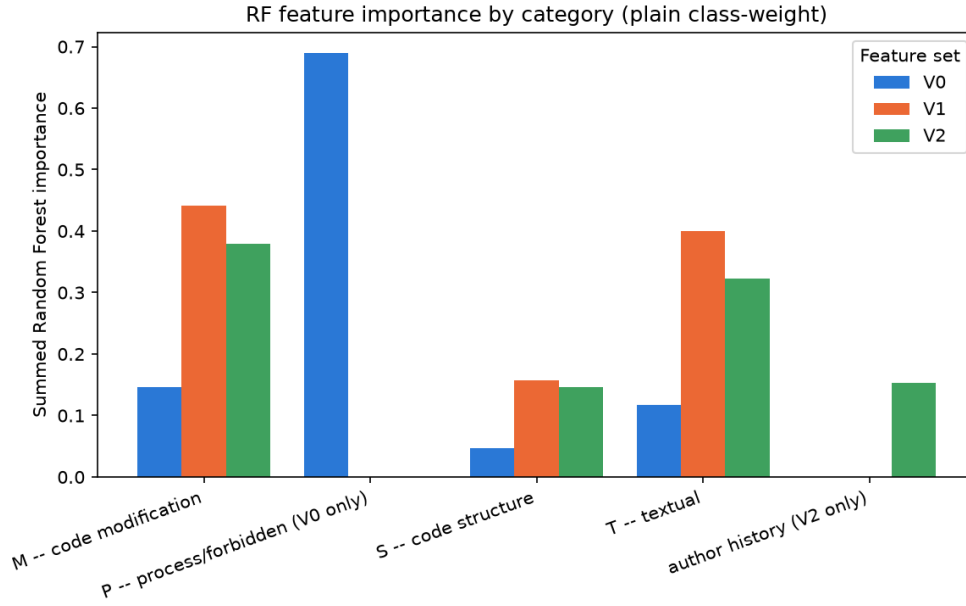


Figure 1. RF feature importance by category, all three variants — visually confirms P’s dominance in V0 and author-history’s exclusive presence in V2.



Figure 2. SVM vs. RF test-set ROC-AUC across V0/V1/V2 (balanced class-weight).

V0 vs. V1 (why §2.4.3 bans process features): ROC-AUC jumps from 0.56–0.68 (V1) to 0.97–0.99 (V0), model-independently. **V1 vs. V2**: V2 *beats* V1 on all 4 matched model/class-weight configurations (+0.06 to +0.16 ROC-AUC) — a genuinely surprising result, since V2 is the *stricter* variant. The two features V2 drops were not pulling much weight in V1 to begin with (neither appears in V1’s top-3 features), while the added `author_prior_merge_rate` is real, non-leaky, and individually predictive — “stricter”

turned out not to mean “worse” here. **SVM vs. RF:** RF’s `class_weight='balanced'` behaves exactly as expected (recall up, accuracy down, e.g. V1 recall(not-merged) 0.250 → 0.425); SVM’s does *not* — on V1 it makes hard-threshold recall *worse* (0.100 → 0.025), plausibly because `CalibratedClassifierCV`’s separate cross-validated calibration pass can partially undo `class_weight`’s shift to the decision function, since the two stages are not jointly optimized.

4 Summary of Problems Encountered

Requirement: Primarily explain how the problems encountered during the experiment were resolved.

1. **A single global time-based cutoff badly conflates repo identity with recency.** Mining samples a recency-biased window per repo (closed PRs, capped, filled with a recent slice), not a uniform scrape of each repo’s full history. A single global 80/20 cutoff on `created_at` collapsed `microsoft/semantic-kernel`’s test slice to **1 PR (of 274)** and pushed test-set merge rate to 87.9% vs. the training split’s 70.8% — an artifact of the sampling window, not a real temporal boundary. *Resolution:* split **per repository** instead (each repo’s own oldest-80%/newest-20%), giving a proportional 890/224 split with a smaller, more plausible train/test merge-rate gap (72.2%/82.1%).
2. **A real `staticfg` bug, not just an “unmaintained library” caveat.** `staticfg`’s `invert()` helper (used to label a CFG’s “false” exit edge) guards a fallback branch with `type(node) == ast.NameConstant` — an attribute Python’s `ast` module has since removed, so *evaluating that comparison itself* raises `AttributeError` for any `if/while` condition that isn’t a direct comparison (`if x:`, `if not x:`, `if x and y:` all crash; `if x > 0:` does not). Since bare-truthiness checks are more idiomatic in real Python than explicit comparisons, this degrades ~27% of otherwise-valid, successfully-parsed Python hunks to `cfg_node_count=0`. *Resolution:* tracked explicitly via separate `parsed_ok/cfg_ok` flags rather than letting a silent CFG-stage crash masquerade as “no control flow.”
3. **A hunk-reconstruction bug from Python’s own `str.split` semantics.** `patch.split("\n")` on a patch string ending in a newline (the normal case) always yields one trailing empty string; an early version of the diff-hunk reconstructor treated that artifact as real unprefixed content and appended it, corrupting the last line of every reconstructed hunk. *Resolution:* caught by a unit test comparing

exact reconstructed output against a hand-built multi-hunk patch string; fixed by explicitly skipping a bare empty line.

4. **pandas.groupby() silently drops NaN group keys by default.** The author-history feature builder grouped by `author_login` to compute each author's prior merge rate; the 3 population PRs with a null `author_login` were silently absent from the grouped iteration entirely (not even receiving the intended cold-start placeholder), so a downstream left-join left *both* `author_prior_merge_rate` and `author_prior_pr_count` as NaN instead of the designed (NaN, 0) signature. *Resolution:* `groupby(..., dropna=False)`, verified against the real affected rows before and after the fix.
5. **SVC(probability=True) is deprecated (scikit-learn 1.9, removed in 1.11).** Surfaced as a `FutureWarning` during a pre-flight smoke test, before the full training run. *Resolution:* switched to the library's own suggested replacement, wrapping the base SVC in `CalibratedClassifierCV` with `ensemble=False` — which later turned out to plausibly explain the SVM class-weight anomaly described in “Comparison of Performance Across Models.”
6. **ColumnTransformer.get_feature_names_out() initially failed.** `FunctionTransformer` (used for the `log1p`/signed-log preprocessing steps) does not implement `get_feature_names_out` by default, which would have made mapping Random Forest's raw `feature_importances_` array back to real column names impossible. *Resolution:* `feature_names_out="one-to-one"` added to each `FunctionTransformer`; the 12 saved models were retrained and their cross-validation scores confirmed bit-identical to before the fix (the change only adds introspection metadata, it does not alter transform output).

5 Reflections on the Experiment

(1) Why is it necessary to construct intermediate code representations such as ASTs and CFGs?

Raw diff text and line-count statistics describe *how much* changed but not *what kind* of change it was structurally. An AST exposes syntactic structure (function/class boundaries, nesting depth) that a flat text diff cannot; a CFG exposes control-flow complexity (branches, loops) directly, which is what a feature like `s_cyclomatic_proxy` actually measures — there is no way to compute a McCabe-style complexity number from raw text alone. Our own results make the payoff concrete: in the compliant V1/V2 feature sets, the S (structure) category still contributes 14.6–15.8% of Random Forest's impor-

tance despite its coverage gaps, meaning it carries real, non-redundant signal beyond what M (line counts) and T (text lengths) already capture.

(2) Why does traditional machine learning require manual feature engineering?

SVM and Random Forest operate on fixed-length numeric feature vectors; unlike a neural network or an LLM, they have no built-in mechanism for learning a representation directly from raw code text or an AST/CFG graph — a human must first decide what numeric summaries to compute and hand-design the categories that hopefully capture the signal (exactly what Steps 8–10 did: freezing an explicit S/M/T column-role taxonomy *before* writing any extraction code). This is also precisely the axis Experiment 3/4 (LLM-based review) will differ on: an LLM consumes the diff and context more directly, without a human first collapsing it into a handful of counts. Our own feature-engineering choices (e.g. the log1p/signed-log/ratio column-role split in `src/ml/common.py`) are themselves evidence of how much manual judgment traditional ML requires before a single model is even trained.

(3) For which application scenarios are SVM and Random Forest respectively suitable?

Textbook answer: SVM (RBF) suits smaller, cleanly-scaled numeric feature spaces where a smooth non-linear boundary is expected; Random Forest suits larger, heterogeneous, less carefully-scaled feature sets and rewards interpretability. Our own evaluation adds a concrete, non-textbook data point: Random Forest’s `class_weight='balanced'` behaved exactly as expected across every variant (minority recall up, accuracy down, e.g. V1 0.250 → 0.425), while SVM’s did not (V1 recall actually *fell*, 0.100 → 0.025) — plausibly because SVM’s probability output here goes through a separate calibration stage (`CalibratedClassifierCV`) that is not jointly optimized with `class_weight`’s effect on the decision boundary. On *this* dataset, at *this* scale, Random Forest is the more predictable choice when class-imbalance handling is the point, independent of which model has the higher raw ROC-AUC.

(4) Which type of feature contributes the most to merge prediction? Why?

M (code modification) and T (textual) contribute roughly equally and dominate in both compliant variants (44.2%/40.0% in V1, 37.9%/32.2% in V2) — because every PR has this information, always fully populated. S (code structure) consistently contributes least (15.8%/14.6%), not because structure is uninformative in principle, but because our own upstream pipeline can only honestly measure it for 71.8% of PRs, and even among those, `staticfg`’s CFG stage itself fails on ~27% of otherwise-valid Python hunks (Problem 2 above) — a concrete, traceable reason rather than a shrug. In V2, a single

non-code feature, `author_prior_merge_rate`, outranks every individual M/T feature (10.0% vs. each individual feature's $\leq 7\%$), showing that legitimate historical/contextual signal can matter more than any one code-derived measurement.

(5) Compared to Experiment 1, what additional steps were included in the data preprocessing for this experiment?

Experiment 1's pipeline was purely descriptive: flatten mined data into tables, compute counts and distributions. Experiment 2 added, in order: (a) filtering to human-written, closed PRs; (b) generating an entirely new representation (AST/CFG) that did not exist in Experiment 1 at all; (c) vectorizing every PR into a fixed-length numeric feature row across an explicit S/M/T/(P) category taxonomy, with missing structural coverage marked honestly rather than imputed as a fake value; (d) column-role-aware preprocessing — `log1p` for heavy-tailed non-negative counts, a sign-preserving log transform for the one feature that can be negative (`m_net_change`), mean-imputation only for already-bounded ratio features, and `StandardScaler` fit strictly on the training split; and (e) a per-repository time-based train/test split (not Experiment 1's aggregate-only statistics) specifically to avoid look-ahead leakage while still keeping every repository proportionally represented. None of (b)–(e) has any equivalent in Experiment 1's pipeline.

Instructor's Comments:

Date: